Aim: Implementation of buffer overflow and its analysis

What is Buffer Overflow?

A buffer overflow occurs when a program writes data beyond the allocated memory buffer's boundaries, overwriting adjacent memory locations. This fundamental memory management flaw has been documented since 1972 in a USAF study on computer security but was not notably exploited until 1988, when the Morris Worm demonstrated the devastating potential of such vulnerabilities. Today, buffer overflow vulnerabilities remain one of the most common and dangerous software vulnerabilities, regularly appearing in CISA Known Exploited Vulnerabilities catalog.

The vulnerability arises from the fundamental design of the C programming language, which provides direct memory access through pointers and performs no automatic array bounds checking. As one security researcher observed, "C is essentially high-level assembly" that "exposes raw pointers to memory and does not perform bounds-checking on arrays". This design choice, while enabling high performance and low-level system control, places the burden of memory safety entirely on the developer.

Memory Layout

Understanding buffer overflow attacks requires comprehension of how programs organize memory. A typical Unix process memory layout consists of several segments:

- **Text (Code) Segment**: Contains executable instructions
- **Data** Segment: Stores global and static variables
- Heap: Dynamically allocated memory growing upward
- Stack: Stores function call information, local variables, and return addresses, growing downward

The stack is particularly relevant to buffer overflow attacks. When a function is called, the compiler generates a stack frame that contains:

- Function arguments
- Return address (where execution resumes after the function completes)
- Saved frame pointer (previous base pointer value)
- Local variables arranged contiguously

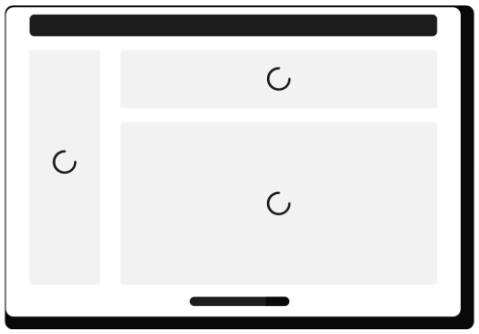
Working

When an attacker provides input larger than the allocated buffer, the excess data flows into adjacent memory regions. In stack-based overflows, this means overwriting:

- **Local variables**: Immediately following the buffer, potentially changing program logic (as demonstrated in our vulnerable login program)
- **Saved** EBP: Corrupting the frame pointer, causing crashes or enabling advanced exploitation techniques
- **Return address**: Allowing attackers to redirect program execution to injected code or existing code segments

Real-World Attack Scenarios

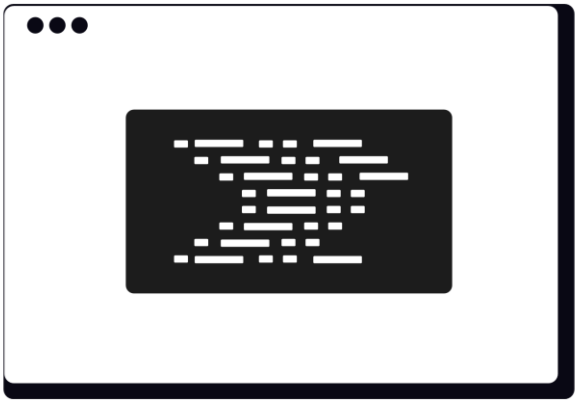
The Morris Worm remains the most famous buffer overflow exploit. Launched on ARPAnet, it exploited a vulnerability in the UNIX `fingerd daemon`. The program accepted input without size limits, enabling the worm to overwrite the return address and execute arbitrary code. Approximately 6,000 servers crashed, representing about 10% of the Internet at the time. The worm's replication mechanism caused it to be sent repeatedly to the same computer, making it difficult to remove and causing widespread disruption.

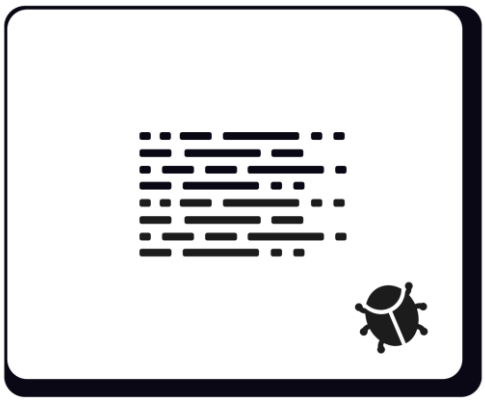


Code

```
#include <stdio.h>
#include <string.h>

int login() {
    char password[16];
    int access = 0;

    printf("Correct password: 'Jack Baker'\n");
    printf("Buffer size: 16 bytes (array size: 17)\n\n");

    printf("Enter password: ");
    gets(password);

    printf("\n--- Memory Analysis ---\n");
    printf("Password stored at: %p\n", password);
    printf("Access flag at:     %p\n", &access);
    printf("Access value:       %d\n", access);
    printf("Password entered:   '%s'\n", password);
    printf("Password length:    %lu\n", strlen(password));
    printf("----------------------\n\n");

    if (strcmp(password, "Jack Baker") == 0) {
        access = 1;
        printf("Password correct!\n");
```

```
    } else {
        printf("Wrong password!\n");
    }

    return access;
}

int main() {

    int result = login();

    if (result == 1) {
        printf("ACCESS GRANTED!\n");
        printf("   You are logged in as a Resident Evil!\n");
    } else {
        printf("ACCESS DENIED!\n");
        printf("   Invalid resident!\n");
    }

    return 0;
}
```





Output

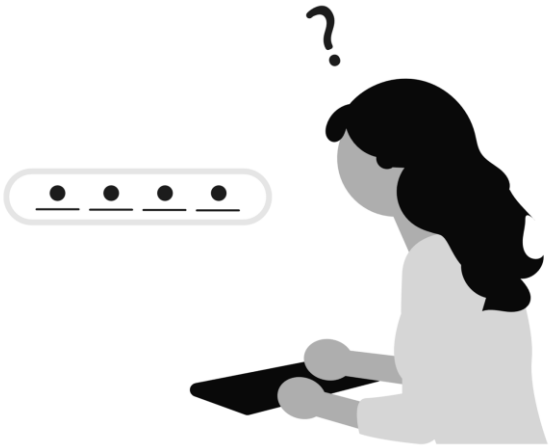
```
spectre@ghouled-gadget MINGW64 ~/Local Volume (D)/degree/SEM 7/SAD/Experiments/output (master)
$ ./SADEXP4.exe
Correct password: 'Jack Baker'
Buffer size: 16 bytes (array size: 17)

Enter password: Jack Baker

--- Memory Analysis ---
Password stored at: 000000D5309FF900
Access flag at:     000000D5309FF8FC
Access value:       0
Password entered:   'Jack Baker'
Password length:    10
----------------------

Password correct!
ACCESS GRANTED!
   You are logged in as a Resident Evil!
```

```
spectre@ghouled-gadget MINGW64 ~/Local Volume (D)/degree/SEM 7/SAD/Experiments/output (master)
$ ./SADEXP4.exe
Correct password: 'Jack Baker'
Buffer size: 16 bytes (array size: 17)

Enter password: Ethan Winters

--- Memory Analysis ---
Password stored at: 0000005174DFFDB0
Access flag at:     0000005174DFFDAC
Access value:       0
Password entered:   'Ethan Winters'
Password length:    13
----------------------

Wrong password!
ACCESS DENIED!
   Invalid resident!
```

Vulnerability Analysis

The critical vulnerability lies in the `gets()` function call. According to `CWE-242` (Use of Inherently Dangerous Function), `gets()` is unsafe because it "reads input without any limit on its size, allowing an attacker to overflow the destination buffer and potentially execute arbitrary code" . The function copies all input from standard input to the buffer without checking size, allowing user-provided strings larger than the buffer size to create overflow conditions .

The stack layout in our program positions the `password` buffer and `access` flag in close proximity. When an attacker provides a password longer than 18 characters, the excess data overwrites the `access` flag. Since `access` is stored immediately after `password` in memory (depending on compiler and architecture), a sufficiently long input will set `access` to a non-zero value, granting unauthorized access regardless of password correctness.



The memory analysis section of our code reveals the addresses of both variables, demonstrating their close proximity on the stack. This is consistent with how compilers organize local variables contiguously in the stack frame .

Prevention

Preventing buffer overflow vulnerabilities requires a multi-layered approach:

- **Language selection**: Prioritize memory-safe languages for new development and migrate critical components over time
- **Secure coding**: Use bounded functions, validate inputs, and avoid dangerous functions
- **Compiler protections**: Enable stack canaries, `ASLR`, and `DEP`

- **Testing**: Employ static analysis, fuzzing, and instrumented testing
- **Organizational commitment**: Develop memory safety roadmaps and conduct root cause analysis

The "`Secure by Design`" principle is essential: manufacturers must take immediate action to prevent these vulnerabilities from being introduced into their products rather than relying on detection and patching after exploitation occurs.

Test Cases

Input Length	Password Buffer	Access Flag	Result
≤ 15 chars	Filled	Unchanged	DENIED
16 chars	Fully filled	Unchanged	DENIED
17-20 chars	Overflow	Overwritten	GRANTED
> 20 chars	Overflow	Overwritten	Crash

Conclusion

The buffer overflow implementation clearly shows how the unsafe `` `gets()` `` function allows input to overwrite the adjacent `` `access` `` flag, granting unauthorized admin access. This vulnerability, identical to those exploited in historic attacks like the Morris Worm and recent incidents such as `CVE-2025-22457`, can be prevented by using bounded functions like `` `fgets()` ``, enabling compiler protections, and adopting memory-safe languages.